Як насправді працюють Full Micro, MicroMacro та Structural A/U–E/U–E/E

Просте послідовне пояснення з однією мережею, формулами та схемами

Що треба зрозуміти з цього тексту

Усі варіанти нижче описують поширення інфекції в мережі, але вони **бачать різну кількість інформації про міжспільнотні контакти**. Full Micro бачить кожне ребро. Original MicroMacro бачить детальну мережу всередині кожної community, але між communities працює з агрегатами. Structural A/U, E/U та E/E — це окремий діагностичний simulator, де ми по одному повертаємо інформацію, втраченою агрегацією.

1. Почнемо з однієї й тієї самої мережі

Нехай є дві communities: A і B. Усередині кожної є звичайна контактна мережа людей. Між communities існують лише кілька реальних контактів. Червоні вузли вже заражені, сірі — susceptible. Синя рамка означає boundary/gateway-вузол, який має контакт з іншою community.

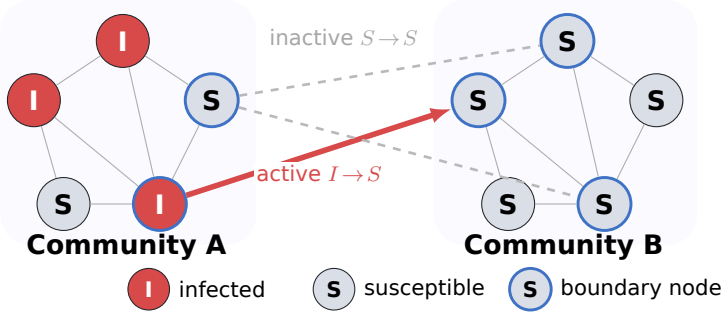


Figure 1: Одна фізична мережа. Між communities є три реальні cross-edges, але зараз активне лише одне $I \to S$ ребро. Саме цю різницю між "усіма ребрами" та "активними ребрами" далі бачать або втрачають різні моделі.

Ця сама мережа буде подана всім симуляторам. Відмінність не в самій картинці, а в тому, **яку частину цієї картинки алгоритм використовує для міжcommunity передачі**.

2. Full Micro: алгоритм бачить усе

Full Micro — це повний Gillespie на всій мережі. Для нього немає принципової різниці між "внутрішнім" і "міжcommunity" ребром. Усі контакти — це звичайні ребра одного графа.

Для кожного ребра (u, v), де u infected, а v susceptible, існує канал зараження з rate

$$a_{uv} = \beta w_{uv} \mathbb{I}[X_u = I,\, X_v = S].$$

Якщо ребра невагові, $w_{uv} = 1$. Загальний infection hazard — це сума rates усіх активних $I \to S$ ребер:

$$\Theta_{\text{micro}} = \sum_{(u,v)\in E} a_{uv}.$$

Тобто Full Micro буквально запитує: *"Які конкретно заражені люди зараз мають контакт із конкретними susceptible людьми?"*

У мережі на рис. 1 між A і B є три фізичні ребра, але активне зараз тільки одне. Якщо всі мають rate β, точний міжcommunity hazard дорівнює

$$\lambda_{AB}^{\text{exact}} = \beta.$$

Коли це ребро спрацьовує, заражається **саме той susceptible-вузол, який стоїть на його кінці**. Full Micro не вигадує target: його визначає реальне ребро.

Інтуїція

Full Micro = "я знаю стан кожної людини, кожне ребро і тому точно знаю, які переходи зараз можливі".

Це відповідає твоєму попередньому exact Gillespie-підходу на contact network: інтенсивність susceptible-вузла визначається його зараженими сусідами, а наступний event вибирається серед реально доступних transitions.

3. Original MicroMacro: детально всередині, грубо між communities

MicroMacro робить головне спрощення: **усередині community він усе ще запускає локальний Gillespie на конкретних вузлах**, але між communities стискає інформацію.

Для macro-рівня community A перетворюється приблизно на такий запис:

$$A: \quad I_A,\ S_A,\ N_A,$$

а пара communities A, B має агреговану вагу W_{AB}, яка описує силу їхнього зв'язку.

У твоїй MoMLeT-моделі міжcommunity rate записується як

$$\lambda_{AB}(t) = \beta\, W_{AB}\, C_A(t)\, S_B(t)\, T,$$

де C_A — агрегована contagiousness source-community, а S_B — агрегована susceptibility destination-community. Для інтуїції це можна читати як схему

$$\lambda_{AB}^{A/U} \propto \beta W_{AB} \frac{I_A}{N_A} \frac{S_B}{N_B}.$$

У нашому toy example $W_{AB} = 3$ (три міжcommunity ребра), $I_A/N_A = 2/5$, а $S_B/N_B = 1$. Тоді агрегована формула може дати hazard близько

$$\lambda_{AB}^{A/U} = 3\beta \cdot \frac{2}{5} = 1.2\beta,$$

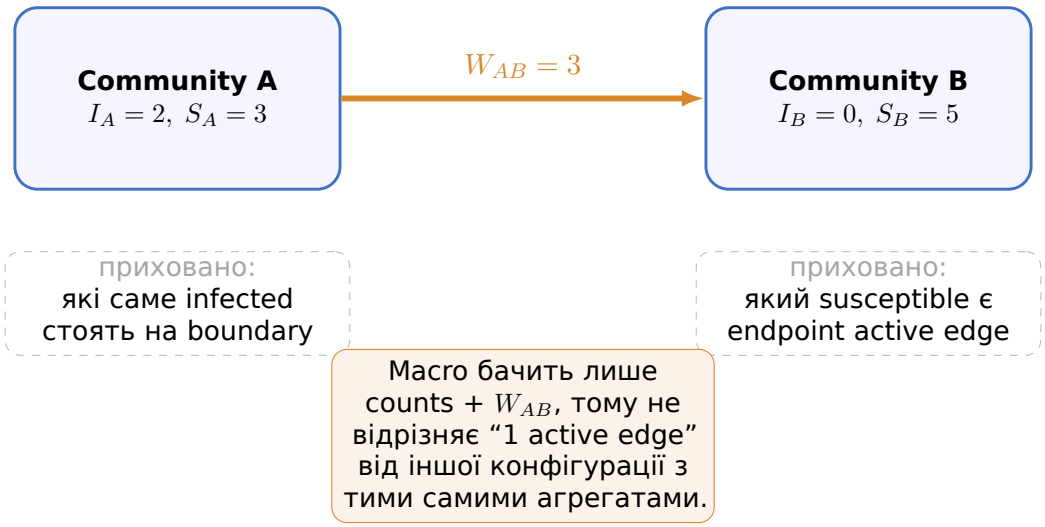


Figure 2: Macro-представлення тієї самої мережі. Локальні вузли не зникають із micro-models, але для міжcommunity hazard production MicroMacro використовує агреговані counts і W_{AB}, а не поточні стани конкретних cross-edges.

хоча Full Micro бачить лише одне реально активне ребро і дає β.

Це і є **структурна апроксимація**: macro-рівень припускає, що infected і susceptible “розмішані” відносно boundary приблизно так само, як у всій community.

Коли macro-event відбувся, original MicroMacro не знає точний endpoint cross-edge. Тому він додає infection у **випадковий susceptible-вузол** destination-community.

3.1. Ще одна різниця: як MicroMacro рухає час

Original MicroMacro не просто має aggregate hazard. Він має власний production scheduler. Micro communities просуваються блоками довжини τ_{micro}, а macro-рівень накопичує hazard

$$H(t) = \int_{t_0}^{t} \Theta(s)\, ds$$

до випадкового порогу $E = -\ln U$. У статті цей інтеграл апроксимується на micro-grid, зокрема через midpoint rate в кожному інтервалі.

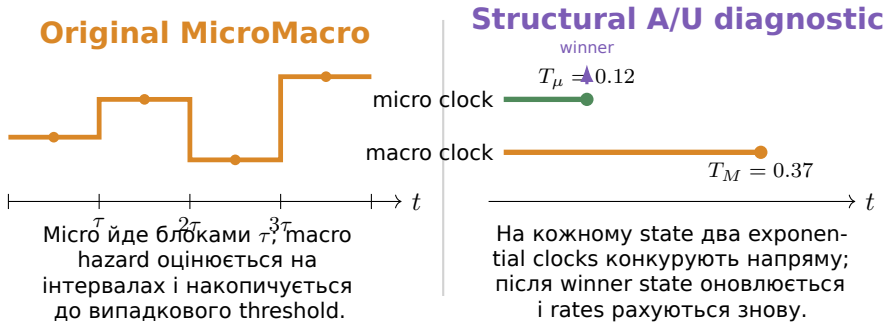


Figure 3: Різниця scheduling. Production MicroMacro використовує τ-синхронізацію та cumulative-hazard integration; A/U diagnostic залишає ту саму aggregate A/U ідею, але використовує direct event competition.

4. Structural A/U: ті самі грубі припущення, але інший simulator

Оце найважливіший момент.

Structural A/U означає:

$$\boxed{A = \text{Aggregate hazard}}, \qquad \boxed{U = \text{Uniform placement}}.$$

Тобто за **структурним змістом** A/U робить те саме грубе припущення, що й original MicroMacro:

- міжcommunity hazard рахується з агрегованої інформації, а не зі стану конкретних cross-edges;
- після import заражається випадковий susceptible у destination-community.

Але A/U — **не той самий executable code**. У diagnostic implementation немає production τ-scheduler у тому самому вигляді. Там micro і macro події конкурують як event-driven stochastic clocks.

Навіщо взагалі потрібен A/U diagnostic?

Від Full Micro original MicroMacro відрізняється одразу у двох місцях. По-перше, він замінює точні міжcommunity active edges на **aggregate hazard** і після import використовує **uniform placement**. По-друге, він використовує власний production τ-scheduler замість прямої глобальної конкуренції подій.
A/U diagnostic залишає першу відмінність, але прибирає другу: **aggregate hazard + uniform placement залишаються, а production τ-scheduler замінено на direct event competition**. Тому порівняння Full Micro з A/U показує переважно структурну похибку самої A/U-апроксимації, без домішки production synchronization mechanics.

Тому різниця така:

	Original MicroMacro	**Structural A/U**
Macro information	aggregate	aggregate
Placement	uniform susceptible	uniform susceptible
Hazard idea	та сама A/U approximation	та сама A/U approximation
Time scheduler	production τ + hazard integral	окремий event-driven scheduler
Executable	original user-facing simulator	окремий diagnostic simulator

Отже відповідь на питання “*чи hazard рахується інакше?*” така:

Коротка відповідь

У A/U **не обов’язково інша structural formula hazard**; за змістом вона така сама aggregate approximation, як у MicroMacro. Головна відмінність A/U від original MicroMacro — **не formula, а simulation scheduler та implementation mechanics**. Саме тому A/U потрібен, щоб подивитися на “чисту” structural approximation без частини production implementation artifacts.

5. E/U: виправляємо тільки hazard

Тепер ми залишаємо uniform placement, але повертаємо точну інформацію про реальні міжcommunity ребра.

Для pair $A \to B$ exact hazard:

$$\lambda_{AB}^{E/U} = \beta \sum_{(u,v)\in E_{AB}} w_{uv}\, \mathbb{I}[X_u = I,\, X_v = S].$$

У toy network з рис. 1 активне тільки одне cross-edge, тому

$$\lambda_{AB}^{E/U} = \beta,$$

а не 1.2β, як у нашому простому aggregate прикладі.

Але після того, як macro-event відбувся, E/U все ще робить "U": вибирає **будь-який susceptible** у B, а не обов'язково endpoint активного ребра.

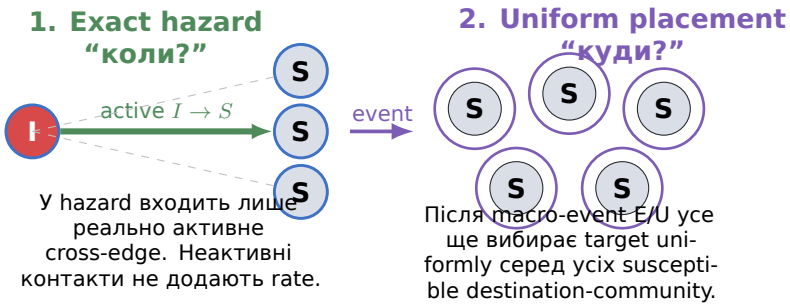


Figure 4: E/U виправляє тільки hazard. Він точно визначає інтенсивність міжcommunity передачі з active edges, але placement після event залишається uniform.

Тому перехід A/U $\to$ E/U вимірює насамперед помилку від **aggregate hazard**. Обидва варіанти запускаються тим самим diagnostic event-driven scheduler і в обох placement залишається uniform. Змінюється лише джерело macro hazard: у A/U він агрегований, а в E/U рахується з реальних active $I \to S$ cross-edges.

Що саме перевіряє E/U?

Якщо E/U стає помітно ближчим до Full Micro, ніж A/U, це означає, що значна частина structural error виникала через **втрату інформації про те, які міжcommunity ребра реально активні**. Scheduler тут не міняється, тому його вплив не плутається з ефектом exact hazard.

6. E/E: виправляємо і hazard, і placement

E/E означає:

$$\boxed{E = \text{Exact hazard}}, \qquad \boxed{E = \text{Exact placement}}.$$

Hazard такий самий exact active-edge sum, як в E/U. Але тепер, якщо event вибрано через конкретне ребро (u, v), заражається саме його endpoint v.

Тобто E/E знає обидві речі:

1. **як часто** має відбуватися inter-community transmission;

2. **куди саме** він може потрапити.

Перехід E/U → E/E ізолює помилку від **uniform placement**. Scheduler і exact hazard залишаються такими самими, як в E/U; змінюється лише правило вибору target-вузла після міжcommunity event.

Що саме перевіряє E/E?

E/E відповідає на питання: "якщо ми вже правильно знаємо *коли* має відбутися міжcommunity зараження, скільки ще похибки дає те, що Micro-Macro не знає *який саме* вузол має заразитися?" Якщо E/E майже збігається з Full Micro, це також є sanity check, що diagnostic event-driven implementation правильно відтворює exact process, коли йому повернули всю потрібну edge-level інформацію.

7. То яка різниця між E/E і Full Micro?

Це тонке, але просте питання.

Full Micro має одну глобальну мережу. Для нього internal та cross-community edges — звичайні ребра одного CTMC. Ніякого спеціального macro-рівня для фізики зараження йому не потрібно.

E/E усе ще написаний як двошаровий diagnostic simulator: internal events і inter-community events організовані окремими блоками логіки. Але для inter-community частини він уже використовує точні active edges та exact endpoint.

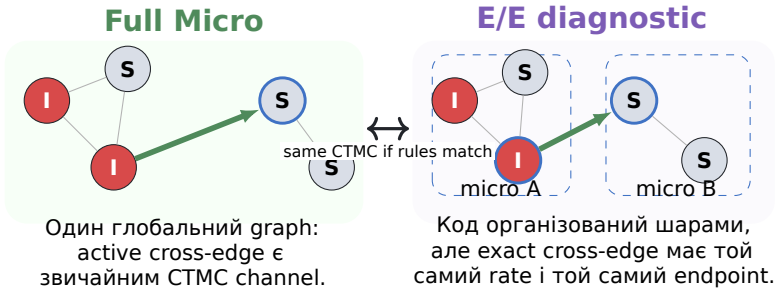


Figure 5: E/E і Full Micro можуть задавати той самий CTMC, хоча програмна архітектура різна. Різниця в організації коду, а не обов'язково в stochastic process.

Якщо одночасно виконуються умови:

- internal Gillespie rules однакові;
- exact cross-edge rates однакові;
- recovery та інші event types однакові;
- exact target визначається тим самим active edge;

то обидва algorithms реалізують той самий CTMC generator. Тоді вони не зобов'язані дати однакову конкретну random trajectory, але повинні давати **той самий статистичний розподіл** при багатьох replications.

І саме тому E/E — хороший sanity check. Якщо E/E не збігається з Full Micro статистично, проблема вже не в A/U approximation, а в тому, що diagnostic implementation не відтворює exact process.

8. Уся логіка A/U → E/U → E/E одним ланцюжком

Перед самим ablation ladder корисно тримати в голові дві різні причини, чому original MicroMacro може відрізнятися від Full Micro:

Structural: exact boundary edges → aggregate community hazard.
Scheduler: global event competition → τ-synchronized hazard integration.

A/U diagnostic спеціально прибирає другу різницю, але залишає першу. Далі A/U → E/U повертає exact hazard, а E/U → E/E повертає exact placement.

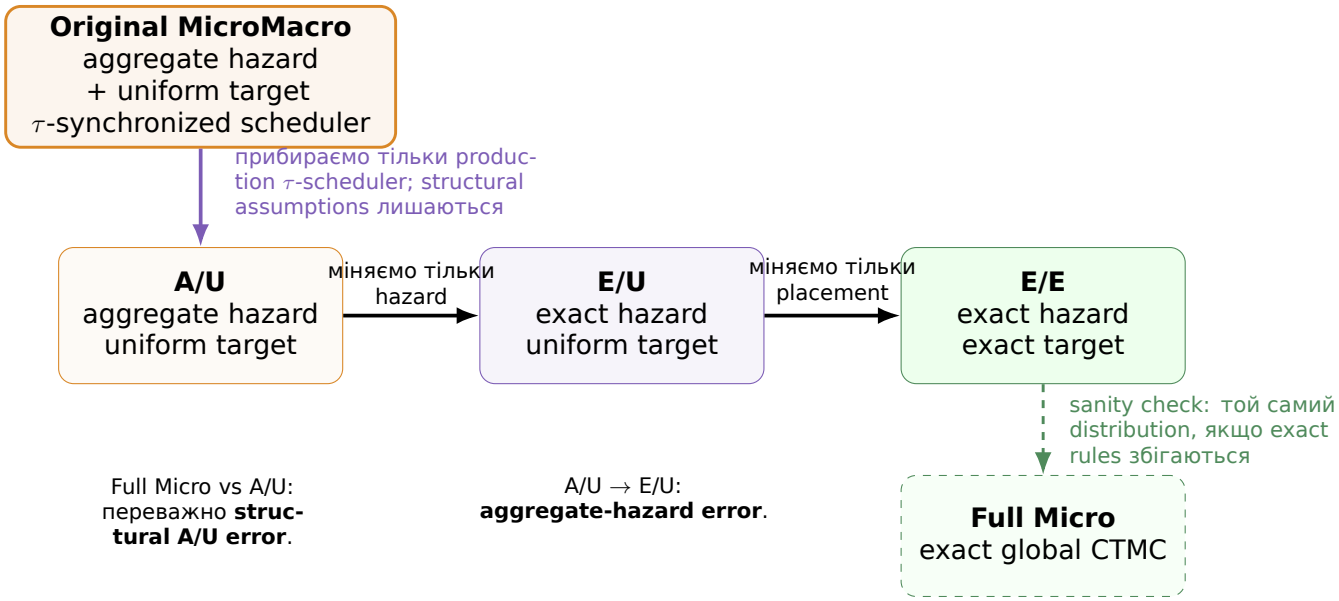


Figure 6: Що саме ізолює кожен крок. A/U відділяє structural assumptions від production scheduling; E/U змінює лише hazard; E/E змінює лише placement і служить sanity check проти Full Micro.

9. Що вважати основним результатом для статті

Тут важливе методологічне правило.

Якщо ми хочемо сказати щось саме про **твій user-facing MicroMacro method**, primary experiments краще робити на original 'MicroMacroSimulator' і змінювати мінімально необхідну деталь:

Experiment	Найчистіша реалізація
A/U baseline	original MicroMacro без змін
E/U	original MicroMacro + замінити лише macro hazard provider на exact active-edge hazard
E/E	те саме + замінити лише target selector на exact edge endpoint
Gateway near/random/far	simulator взагалі не змінювати; змінювати тільки topology
$d \times k$ grid	simulator не змінювати; змінювати лише параметри мережі

Окремий ‘HazardPlacementSimulator‘ все одно корисний: він дуже добре підходить як **secondary mechanistic diagnostic**, бо прибирає production scheduler artifacts і дозволяє чисто розкласти structural error. Але його результати краще подавати як підтвердження механізму, а не як єдине свідчення того, що отримає користувач original MicroMacro.

10. Найкоротше резюме

Full Micro: бачить кожен вузол і кожне active edge. Exact hazard, exact target.

Original MicroMacro: усередині community бачить nodes, між communities бачить aggregate I, S, W. Aggregate hazard, uniform target, production τ-scheduler.

Structural A/U: ті самі aggregate/uniform structural assumptions, але окремий event-driven diagnostic simulator; тому він прибирає production τ-scheduler і залишає structural A/U approximation.

E/U: той самий diagnostic scheduler і той самий uniform placement, але aggregate hazard замінено на exact cross-edge hazard; різниця A/U $\rightarrow$ E/U ізолює hazard information loss.

E/E: той самий diagnostic scheduler та exact hazard, але uniform placement замінено на exact endpoint; різниця E/U $\rightarrow$ E/E ізолює placement error. Якщо решта event rules збігаються з Full Micro, статистично має відтворювати той самий CTMC.

Примітка про джерела

Опис original MicroMacro, його двох рівнів, macro-rate та cumulative-hazard synchronization базується на наданій роботі *Towards complexity reduction of large-scale epidemic simulation in two-scale networks* (MoMLeT-2025). Опис Full Micro спирається на надану роботу *Simulating epidemic peak dynamics on complex networks using efficient Gillespie algorithms*. Розрізнення Structural A/U–E/U–E/E як окремої diagnostic implementation базується на перевіреному source code експериментів у Google Drive в межах поточного аудиту.